

Đánh giá nền tảng ANN, CNN, AlexNet và lộ trình học Reinforcement Learning

Mục lục

1	Kết luận	2
2	Những gì bạn đã học có ích thế nào cho RL?	2
2.1	ANN và Backpropagation: rất hữu ích cho Deep RL	2
2.2	CNN: chỉ cần trong một số bài toán RL	2
2.3	AlexNet: không phải điều kiện tiên quyết	3
3	Các phần trong lộ trình hiện tại: phần nào cần cho RL?	3
4	Những kiến thức còn thiếu nhưng sheet chưa đề cập đầy đủ	4
4.1	Xác suất và kỳ vọng — bắt buộc	4
4.2	Markov Decision Process — bắt buộc	5
4.3	Value function và Bellman equation — bắt buộc	7
4.4	Dynamic Programming — rất nên học trước Q-Learning	9
4.5	Exploration và exploitation — bắt buộc	11
4.6	Monte Carlo, Temporal Difference và SARSA — nên biết	13
4.7	Python, NumPy và môi trường RL — cần cho thực hành	15
5	Điều kiện để chuyển từ RL cơ bản sang Deep RL	18
6	Lộ trình phù hợp nhất với tình trạng hiện tại	19
6.1	Giai đoạn 1 — Bổ sung nền tảng tối thiểu	19
6.2	Giai đoạn 2 — RL dạng bảng	20
6.3	Giai đoạn 3 — Deep RL	20
7	Checklist sẵn sàng	21

1 Kết luận

Bạn đã đủ nền tảng để bắt đầu học RL ở mức nhập môn, đặc biệt là **tabular Q-Learning**. Tuy nhiên, ANN/CNN/AlexNet không phải các kiến thức nền trực tiếp quan trọng nhất của RL.

Bạn chưa nên nhảy thẳng vào **Deep Reinforcement Learning** như DQN, PPO hoặc **Actor-Critic**. Trước tiên cần bổ sung một số kiến thức về xác suất, quá trình Markov và các thuật toán RL cơ bản.

Theo lộ trình hiện tại, bạn đã hoàn thành phần lớn **ANN/Backpropagation, CNN** và **AlexNet 1**; trong khi các nội dung **Q-Learning** vẫn chưa học.

2 Những gì bạn đã học có ích thế nào cho RL?

2.1 ANN và Backpropagation: rất hữu ích cho Deep RL

Bạn đã học:

- Cấu trúc mạng neural.
- Forward propagation và backpropagation.
- Gradient descent.
- Softmax.
- Vanishing gradient.
- Underfitting và overfitting.

Đây là nền tảng tốt để sau này hiểu:

- Mạng Q trong DQN.
- Policy network.
- Value network.
- Actor và Critic.
- Cách tối ưu loss trong Deep RL.

2.2 CNN: chỉ cần trong một số bài toán RL

CNN hữu ích khi trạng thái của môi trường là hình ảnh, chẳng hạn:

- Chơi game Atari từ khung hình.

- Điều khiển robot bằng camera.
- Xe tự hành.
- Điều hướng dựa trên hình ảnh.

Với các môi trường ban đầu như FrozenLake, Taxi, CartPole hoặc MountainCar, bạn **không cần CNN**.

2.3 AlexNet: không phải điều kiện tiên quyết

AlexNet giúp bạn hiểu một kiến trúc CNN cụ thể, nhưng không giúp nhiều cho RL nhập môn. Bạn không cần hoàn thành AlexNet 2 hoặc phần Matlab AlexNet trước khi học RL.

3 Các phần trong lộ trình hiện tại: phần nào cần cho RL?

Chủ đề trong sheet	Có cần trước RL không?	Nhận xét
Supervised và Unsupervised Learning	Nên biết	Cần hiểu RL khác supervised learning ở điểm nào.
Linear Regression	Nên học	Hữu ích để hiểu function approximation, prediction và loss.
KNN	Không bắt buộc	Có thể bỏ qua trước khi học RL.
K-Means	Không bắt buộc	Không liên quan trực tiếp đến RL cơ bản.
SVM	Không bắt buộc	Không phải nền tảng của RL.
Perceptron	Không cần học lại	ANN bạn đã học đã bao hàm ý tưởng này.
AlexNet 2	Không bắt buộc	Chỉ cần khi quan tâm đến CNN chuyên sâu.
Q-Learning	Bắt buộc	Đây là điểm bắt đầu trực tiếp của RL.
RNN/LSTM	Chưa cần	Chỉ cần sau này với trạng thái không quan sát đầy đủ hoặc dữ liệu chuỗi.
Genetic Algorithm	Không bắt buộc	Là hướng tối ưu khác, không phải nền tảng RL.

Như vậy, bạn **không cần quay lại học toàn bộ KNN, K-Means và SVM rồi mới bắt đầu RL**.

Trong các nội dung chưa học của sheet, phần nên bổ sung nhất là:

1. Khái niệm supervised, unsupervised và reinforcement learning.
2. Linear regression ở mức cơ bản.
3. Các nội dung Q-Learning.

4 Những kiến thức còn thiếu nhưng sheet chưa đề cập đầy đủ

4.1 Xác suất và kỳ vọng — bắt buộc

Bạn cần hiểu:

- Biến ngẫu nhiên.
- Xác suất có điều kiện.
- Phân phối xác suất.
- Kỳ vọng $\mathbb{E}[X]$.
- Trung bình có trọng số theo xác suất.
- Sampling.
- Variance ở mức cơ bản.

Trong RL, phần thưởng và trạng thái tiếp theo thường mang tính ngẫu nhiên. Giá trị của một trạng thái thực chất là **kỳ vọng của tổng phần thưởng trong tương lai**:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s \right]. \quad (1)$$

Nếu chưa hiểu kỳ vọng, công thức value function và Bellman equation sẽ khá khó tiếp cận.

Lý thuyết cơ bản:

Xác suất có điều kiện:

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (2)$$

Luật xác suất toàn phần:

$$P(A) = \sum_i P(A|B_i)P(B_i) \quad (3)$$

Kỳ vọng của biến ngẫu nhiên rời rạc:

$$\mathbb{E}[X] = \sum_x x \cdot P(X = x) \quad (4)$$

Kỳ vọng của biến ngẫu nhiên liên tục:

$$\mathbb{E}[X] = \int_{-\infty}^{\infty} x \cdot f_X(x) dx \quad (5)$$

Tính chất kỳ vọng:

$$\mathbb{E}[aX + bY] = a\mathbb{E}[X] + b\mathbb{E}[Y] \quad (6)$$

Variance:

$$\text{Var}(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \quad (7)$$

Ví dụ 1: Giả sử agent ở trạng thái s và có 3 hành động có thể thực hiện. Hành động a_1 có 70% xác suất nhận reward +10 và 30% xác suất nhận -5. Kỳ vọng reward của a_1 là:

$$\mathbb{E}[R|a_1] = 0.7 \times 10 + 0.3 \times (-5) = 7 - 1.5 = 5.5 \quad (8)$$

Agent cần so sánh kỳ vọng này với các hành động khác để quyết định tối ưu.

Ví dụ 2 (Return trong RL): Agent thực hiện 4 bước với rewards: $R_1 = 1, R_2 = 0, R_3 = 5, R_4 = 10$. Với $\gamma = 0.9$, return tại bước đầu tiên:

$$G_0 = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 = 1 + 0.9(0) + 0.81(5) + 0.729(10) = 1 + 0 + 4.05 + 7.29 = 12.34 \quad (9)$$

Nếu môi trường stochastic, agent phải tính $\mathbb{E}[G_0]$ trên nhiều trajectories khác nhau.

Tài liệu tham khảo:

- *Sutton & Barto, Reinforcement Learning: An Introduction (2nd ed.)*, Chapter 3 — giới thiệu expected return và vai trò của kỳ vọng trong RL.
- *Khan Academy — Probability and Statistics* (<https://www.khanacademy.org/math/statistics-probability>) — nền tảng xác suất và kỳ vọng.
- *MIT 18.05 Introduction to Probability and Statistics* — khóa học xác suất cơ bản với ứng dụng.

4.2 Markov Decision Process — bắt buộc

Đây là nền tảng quan trọng nhất của RL.

Bạn cần phân biệt rõ:

- **Agent:** tác nhân học.
- **Environment:** môi trường.
- **State s :** trạng thái.
- **Action a :** hành động.
- **Reward r :** phần thưởng.
- **Transition:** cách môi trường chuyển trạng thái.
- **Policy $\pi(a | s)$:** chiến lược chọn hành động.
- **Episode:** một lượt tương tác hoàn chỉnh.
- **Terminal state:** trạng thái kết thúc.

- **Discount factor** γ : mức coi trọng phần thưởng tương lai.
- **Return** G_t : tổng phần thưởng tích lũy.

Bạn cũng cần hiểu **Markov property**:

Khi biết trạng thái hiện tại, tương lai không phụ thuộc trực tiếp vào toàn bộ lịch sử trước đó.

Lý thuyết cơ bản:

MDP được định nghĩa bởi tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$:

- $\mathcal{S}$: tập trạng thái (state space)
- $\mathcal{A}$: tập hành động (action space)
- $P(s'|s, a)$: xác suất chuyển trạng thái (transition probability)
- $R(s, a, s')$: hàm reward
- $\gamma \in [0, 1]$: discount factor

Dynamics của môi trường:

$$p(s', r|s, a) = P(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a) \quad (10)$$

Markov property:

$$P(S_{t+1} = s', R_{t+1} = r \mid S_0, A_0, R_1, \dots, S_t, A_t) = P(S_{t+1} = s', R_{t+1} = r \mid S_t, A_t) \quad (11)$$

Tương lai chỉ phụ thuộc vào hiện tại, không phụ thuộc lịch sử.

Return (tổng reward tích lũy):

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \quad (12)$$

Policy: ánh xạ từ trạng thái sang xác suất hành động

$$\pi(a|s) = P(A_t = a \mid S_t = s) \quad (13)$$

Deterministic policy: $a = \pi(s)$. Stochastic policy: $a \sim \pi(\cdot|s)$.

Ví dụ 1 — GridWorld 4×4: Agent đứng tại ô (2, 3). Có 4 hành động: lên/xuống/trái/phải. Mỗi bước nhận -1 reward. Đích là góc (4, 4) với reward 0 và kết thúc episode. Đây là MDP hữu hạn với:

- $\mathcal{S}$: 16 trạng thái (ô vuông)
- $\mathcal{A}$: 4 hành động

- $R(s, a, s')$: -1 mỗi bước, 0 tại đích
- γ : thường dùng 0.9 hoặc 0.99

Policy tối ưu π^* sẽ chọn đường ngắn nhất đến đích. Nếu đứng ở $(2, 3)$ và đích ở $(4, 4)$: đi phải 2 bước, đi xuống 1 bước $\Rightarrow$ total return $G = -1 + 0.9(-1) + 0.81(0) = -1.9$.

Ví dụ 2 — Recycling Robot: Robot thu gom rác, có 2 trạng thái năng lượng: high/low. Hành động: search (tìm rác), wait (chờ), recharge (sạc pin). Transition stochastic:

- Từ high + search: 70% ở lại high ($r = 4$), 30% xuống low ($r = 4$)
- Từ low + search: 100% depleted ($r = -3$, phải recharge)
- Từ low + recharge: 100% lên high ($r = 0$)

Đây là MDP stochastic. Policy phải cân bằng giữa thu thập reward và duy trì pin.

Tài liệu tham khảo:

- *Sutton & Barto (2nd ed.)*, Chapter 3 — định nghĩa đầy đủ MDP, return, discount factor.
- *David Silver's RL Course, Lecture 2: Markov Decision Processes* (<https://www.youtube.com/watch?v=1fHX2hHRMVQ>) — bài giảng kinh điển về MDP.
- *Berkeley CS285, Lecture 2* — giải thích Markov property và dynamics.

4.3 Value function và Bellman equation — bắt buộc

Cần hiểu ba đại lượng:

$$V^\pi(s), \quad (14)$$

là giá trị của trạng thái khi đi theo policy π .

$$Q^\pi(s, a), \quad (15)$$

là giá trị của việc thực hiện hành động a tại trạng thái s , sau đó đi theo policy π .

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s), \quad (16)$$

là mức độ hành động a tốt hơn hoặc kém hơn mức trung bình tại trạng thái s .

Bellman equation diễn tả ý tưởng:

$$\text{Giá trị hiện tại} = \text{phần thưởng trước mắt} + \text{giá trị tương lai}. \quad (17)$$

Đây là cầu nối trực tiếp dẫn đến Q-Learning.

Lý thuyết cơ bản:

State-value function: kỳ vọng return từ trạng thái s theo policy π

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right] \quad (18)$$

Action-value function: kỳ vọng return từ trạng thái s khi thực hiện hành động a , sau đó theo π

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right] \quad (19)$$

Advantage function: đo lường hành động a tốt hơn trung bình bao nhiêu

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (20)$$

Bellman Expectation Equation cho V^π :

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V^\pi(s')] \quad (21)$$

Phiên bản đơn giản hóa:

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a) \quad (22)$$

$$Q^\pi(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma V^\pi(s')] \quad (23)$$

Bellman Optimality Equation:

Optimal value function:

$$V^*(s) = \max_{\pi} V^\pi(s) = \max_a Q^*(s, a) \quad (24)$$

$$Q^*(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma V^*(s')] = \sum_{s', r} p(s', r|s, a) \left[r + \gamma \max_{a'} Q^*(s', a') \right] \quad (25)$$

Optimal policy:

$$\pi^*(a|s) = \begin{cases} 1 & \text{nếu } a = \arg \max_a Q^*(s, a) \\ 0 & \text{ngược lại} \end{cases} \quad (26)$$

Ví dụ 1 — FrozenLake 4×4: Agent đứng tại ô an toàn (2, 2) bên cạnh hố băng (2, 3). Nếu rơi xuống hố: $V(\text{hole}) = 0$ (terminal). Nếu đi sang ô khác: nhận $r = 0$ và tiếp tục. Tại trạng thái hiện tại $s = (2, 2)$ với random policy $\pi(\text{up}|s) = \pi(\text{down}|s) = \pi(\text{left}|s) = \pi(\text{right}|s) = 0.25$:

$$V^\pi(s) = 0.25 \cdot [0 + 0.9V(s_{\text{up}})] + 0.25 \cdot [0 + 0.9 \cdot 0] + 0.25 \cdot [0 + 0.9V(s_{\text{left}})] + 0.25 \cdot [0 + 0.9V(s_{\text{down}})] \quad (27)$$

Nếu đi sang phải rơi xuống hố, giá trị tương lai là 0. Policy tốt sẽ tránh hướng đó: $\pi(\text{right}|s) = 0$.

Ví dụ 2 — Tính Q và A : Giả sử tại trạng thái s , $V^\pi(s) = 10$. Có 3 hành động:

- $Q^\pi(s, a_1) = 12 \Rightarrow A^\pi(s, a_1) = 12 - 10 = +2$ (tốt hơn trung bình)
- $Q^\pi(s, a_2) = 10 \Rightarrow A^\pi(s, a_2) = 10 - 10 = 0$ (trung bình)
- $Q^\pi(s, a_3) = 7 \Rightarrow A^\pi(s, a_3) = 7 - 10 = -3$ (kém hơn trung bình)

Policy improvement: tăng $\pi(a_1|s)$, giảm $\pi(a_3|s)$.

Tài liệu tham khảo:

- *Sutton & Barto (2nd ed.)*, Chapter 3.5–3.6 — Bellman equation và optimal policies.
- *David Silver Lecture 2* — giải thích trực quan Bellman backup diagram.
- *OpenAI Spinning Up: Introduction to RL* (https://spinningup.openai.com/en/latest/spinningup/rl_intro.html) — giải thích value function, Bellman equation với code minh họa.

4.4 Dynamic Programming — rất nên học trước Q-Learning

Nên biết:

- Policy evaluation.
- Policy improvement.
- Policy iteration.
- Value iteration.

Dynamic Programming giả sử đã biết mô hình môi trường. Q-Learning thì không cần biết trước mô hình, nhưng cách cập nhật Q-Learning bắt nguồn từ Bellman optimality equation.

Lý thuyết cơ bản:

Dynamic Programming giải MDP khi biết đầy đủ dynamics $p(s', r|s, a)$. Gồm 2 thuật toán chính:

1. *Policy Iteration*: gồm 2 bước lặp cho đến hội tụ:

Policy Evaluation: tính $V^\pi(s)$ cho tất cả trạng thái:

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma V_k(s')] \quad (28)$$

Lặp cho đến khi $\max_s |V_{k+1}(s) - V_k(s)| < \theta$ (ngưỡng hội tụ nhỏ, ví dụ 10^{-4}).

Policy Improvement: cập nhật policy bằng cách chọn greedy theo V :

$$\pi'(s) = \arg \max_a \sum_{s', r} p(s', r|s, a) [r + \gamma V^\pi(s')] = \arg \max_a Q^\pi(s, a) \quad (29)$$

Nếu $\pi' = \pi$ thì đã đạt optimal π^* . Nếu chưa, gán $\pi \leftarrow \pi'$ và quay lại policy evaluation.

2. *Value Iteration*: kết hợp policy improvement vào mỗi bước, tính trực tiếp $V^*(s)$:

$$V_{k+1}(s) = \max_a \sum_{s',r} p(s', r|s, a) [r + \gamma V_k(s')] \quad (30)$$

Sau khi hội tụ, trích xuất optimal policy:

$$\pi^*(s) = \arg \max_a \sum_{s',r} p(s', r|s, a) [r + \gamma V^*(s')] \quad (31)$$

Value iteration thường hội tụ nhanh hơn policy iteration nhưng mỗi vòng lặp tốn thời gian hơn.

Ví dụ 1 — GridWorld 2×2:

Layout:

```
[S]  [ ]
[ ]  [G]
```

S : start, G : goal (terminal, $r = +1$). Reward mỗi bước: -0.01 . Hành động: up/down/left/right. Deterministic transitions. $\gamma = 0.9$.

Khởi tạo: $V_0(S) = 0$, $V_0(\text{middle-right}) = 0$, $V_0(\text{bottom-left}) = 0$, $V_0(G) = 0$.

Value Iteration Iteration 1:

- $V_1(G) = 0$ (terminal)
- $V_1(\text{middle-right}) = \max\{-0.01 + 0.9 \times 0, \dots, -0.01 + 0.9 \times 0 + 1\} = \max\{\dots, 0.99\} = 0.99$ (đi down tới G)
- $V_1(\text{bottom-left})$: tương tự, best action dẫn đến middle-right hoặc gần G
- $V_1(S)$: giá trị từ các trạng thái lân cận

Sau 10–20 vòng lặp, V hội tụ. Policy tối ưu từ S : right $\rightarrow$ down $\rightarrow$ G.

Ví dụ 2 — Policy Iteration trên Recycling Robot:

Khởi tạo random policy: $\pi_0(\text{search}|\text{high}) = 0.5$, $\pi_0(\text{wait}|\text{high}) = 0.5$.

Policy Evaluation: giải hệ phương trình Bellman cho V^{π_0} :

$$V^{\pi_0}(\text{high}) = 0.5 \cdot [4 + 0.7 \times 0.9V(\text{high}) + 0.3 \times 0.9V(\text{low})] + 0.5 \cdot [1 + 0.9V(\text{high})] \quad (32)$$

Giải hệ này cho $V(\text{high})$ và $V(\text{low})$.

Policy Improvement: với V mới, tính $Q(\text{high}, \text{search})$ và $Q(\text{high}, \text{wait})$. Nếu $Q(\text{high}, \text{search}) > Q(\text{high}, \text{wait})$, cập nhật $\pi_1(\text{search}|\text{high}) = 1$. Lặp lại cho đến policy không thay đổi.

Tài liệu tham khảo:

- *Sutton & Barto (2nd ed.)*, Chapter 4 — Dynamic Programming với policy iteration và value iteration.
- *David Silver Lecture 3: Planning by Dynamic Programming* (<https://www.youtube.com/watch?v=Nd1-UUMVfz4>).
- *Berkeley CS285 Lecture 8* — giải thích policy iteration và value iteration.

4.5 Exploration và exploitation — bắt buộc

Agent phải cân bằng giữa:

- **Exploitation**: chọn hành động đang được cho là tốt nhất.
- **Exploration**: thử hành động khác để tìm lựa chọn tốt hơn.

Cần hiểu ít nhất:

- Random policy.
- Greedy policy.
- ϵ -greedy.
- Cách giảm dần epsilon.
- Vì sao chỉ chọn hành động tốt nhất hiện tại có thể khiến agent mắc kẹt.

Lý thuyết cơ bản:

Exploration-exploitation tradeoff là vấn đề cốt lõi của RL. Agent phải quyết định:

- **Exploit**: chọn hành động có Q value cao nhất hiện tại để tối đa hóa reward ngay lập tức.
- **Explore**: thử hành động ít được chọn để cập nhật ước lượng Q , có thể tìm được hành động tốt hơn.

Greedy policy: luôn exploit

$$\pi(s) = \arg \max_a Q(s, a) \quad (33)$$

Nhược điểm: nếu ước lượng Q ban đầu không chính xác, agent sẽ stuck ở local optimum.

Random policy: luôn explore

$$\pi(a|s) = \frac{1}{|\mathcal{A}|} \quad \forall a \quad (34)$$

Nhược điểm: không tận dụng kiến thức đã học, return thấp.

ϵ -greedy policy: cân bằng explore-exploit

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}|} & \text{nếu } a = \arg \max_{a'} Q(s, a') \\ \frac{\epsilon}{|\mathcal{A}|} & \text{nếu } a \neq \arg \max_{a'} Q(s, a') \end{cases} \quad (35)$$

Với xác suất $1 - \epsilon$: chọn hành động tốt nhất (exploit). Với xác suất ϵ : chọn ngẫu nhiên (explore).

Boltzmann exploration (softmax):

$$\pi(a|s) = \frac{\exp(Q(s, a)/\tau)}{\sum_{a'} \exp(Q(s, a')/\tau)} \quad (36)$$

τ (temperature): lớn $\rightarrow$ gần random, nhỏ $\rightarrow$ gần greedy.

Upper Confidence Bound (UCB): chọn hành động có upper confidence cao nhất:

$$a_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right] \quad (37)$$

$N_t(a)$: số lần chọn a . Term thứ hai khuyến khích chọn hành động ít được thử.

Ví dụ 1 — Multi-armed Bandit:

Có 3 máy đánh bạc (arms), true expected rewards: $Q^*(a_1) = 1.0, Q^*(a_2) = 2.5, Q^*(a_3) = 1.5$ (chưa biết). Ban đầu: $Q_0(a_1) = Q_0(a_2) = Q_0(a_3) = 0$.

Lượt 1: thử a_1 , nhận $r = 1.2$. Cập nhật $Q(a_1) = 1.2$.

Lượt 2 với $\epsilon = 0.1$:

- Xác suất 0.9: chọn a_1 (có Q cao nhất)
- Xác suất 0.1: chọn ngẫu nhiên, có thể thử a_2 hoặc a_3

Giả sử thử a_2 , nhận $r = 3.0$. Cập nhật $Q(a_2) = 3.0$ (cao nhất).

Lượt 3–1000: phần lớn chọn a_2 (exploit), thỉnh thoảng thử a_1, a_3 (explore).

Nếu greedy thuần túy: có thể stuck ở a_1 và không bao giờ khám phá a_2 tốt hơn.

Ví dụ 2 — Epsilon decay schedule:

$$\epsilon_t = \max(\epsilon_{\min}, \epsilon_{\text{start}} \times \text{decay}^t) \quad (38)$$

Với $\epsilon_{\text{start}} = 1.0, \text{decay} = 0.995, \epsilon_{\min} = 0.01$:

- Episode 1: $\epsilon = 1.0$ (pure exploration)
- Episode 100: $\epsilon \approx 0.606$
- Episode 500: $\epsilon \approx 0.082$
- Episode 1000: $\epsilon = 0.01$ (mostly exploitation)

Chiến lược này giúp agent explore nhiều lúc đầu để học môi trường, sau đó exploit để tối ưu performance.

Tài liệu tham khảo:

- *Sutton & Barto (2nd ed.)*, Chapter 2 — Multi-armed Bandits và exploration strategies.
- *David Silver Lecture 4: Model-Free Prediction* — giải thích exploration trong context RL.
- *OpenAI Spinning Up: Exploration strategies* — so sánh ϵ -greedy, Boltzmann, UCB.

4.6 Monte Carlo, Temporal Difference và SARSA — nên biết

Không nên chỉ học duy nhất Q-Learning. Trình tự hợp lý là:

1. Multi-armed bandit.
2. Monte Carlo prediction.
3. Temporal Difference, đặc biệt là TD(0).
4. SARSA.
5. Q-Learning.

Lý thuyết cơ bản:

Dynamic Programming cần biết trước model $p(s', r|s, a)$. Model-free methods học trực tiếp từ experience mà không cần model.

1. *Monte Carlo (MC)*: Học từ complete episodes. Cập nhật $V(s)$ sau khi episode kết thúc bằng return thực tế G_t :

$$V(s) \leftarrow V(s) + \alpha[G_t - V(s)] \quad (39)$$

với $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots + R_T$ (T: terminal step).

First-visit MC: chỉ cập nhật lần đầu tiên gặp s trong episode.

Every-visit MC: cập nhật mọi lần gặp s .

Ưu điểm: không bias (ước lượng G_t chính xác). Nhược điểm: phải đợi episode kết thúc, high variance.

2. *Temporal Difference TD(0)*: Học sau mỗi bước. Cập nhật $V(s)$ bằng bootstrapping từ $V(s')$:

$$V(s) \leftarrow V(s) + \alpha \underbrace{[r + \gamma V(s') - V(s)]}_{\text{TD target}} \quad (40)$$

$$\text{TD error: } \delta_t = r + \gamma V(s') - V(s) \quad (41)$$

Ưu điểm: học online (không cần chờ episode kết thúc), low variance. Nhược điểm: có bias (bootstrap từ ước lượng $V(s')$ chưa chính xác).

3. *SARSA (State-Action-Reward-State-Action)* — *on-policy TD control*:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \quad (42)$$

SARSA cập nhật dựa trên hành động a_{t+1} **thực sự thực hiện** theo policy hiện tại (on-policy). Algorithm:

1. Khởi tạo $Q(s, a)$ tùy ý

2. Lặp mỗi episode:
3. Khởi tạo s
4. Chọn a từ s dùng policy dẫn xuất từ Q (e.g., ϵ -greedy)
5. Lặp mỗi bước:
6. Thực hiện a , quan sát r, s'
7. Chọn a' từ s' dùng policy dẫn xuất từ Q
8. $Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma Q(s', a') - Q(s, a)]$
9. $s \leftarrow s'; a \leftarrow a'$
10. cho đến s terminal

4. *Q-Learning — off-policy TD control:*

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right] \quad (43)$$

$$\underbrace{r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)}_{\text{TD error } \delta_t} \quad (44)$$

Q-Learning cập nhật bằng $\max_a Q(s_{t+1}, a)$ bất kể hành động thực tế là gì (off-policy) — nó học policy tối ưu trực tiếp. Behavior policy có thể là ϵ -greedy (explore), nhưng target policy là greedy (exploit).

So sánh MC vs TD vs DP:

	DP	MC	TD
Model	Cần	Không cần	Không cần
Bootstrap	Có	Không	Có
Đợi episode kết thúc	Không	Có	Không
Bias	Không	Không	Có
Variance	Thấp	Cao	Trung bình

Ví dụ 1 — FrozenLake 4×4 với MC:

Episode 1: $S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow \text{hole (fail)}$. Rewards: 0, 0, 0. $G_0 = 0$. Cập nhật $V(S_0) \leftarrow V(S_0) + \alpha[0 - V(S_0)]$ (giảm).

Episode 2: $S_0 \rightarrow S_4 \rightarrow S_8 \rightarrow \dots \rightarrow \text{goal}$. Rewards: 0, 0, ..., 1. $G_0 = \gamma^{10} \times 1 \approx 0.35$. Cập nhật $V(S_0)$ tăng lên.

Sau nhiều episodes, $V(S_0)$ hội tụ đến giá trị trung bình.

Ví dụ 2 — TD(0) trên cùng trajectory:

Bước 1: $S_0 \rightarrow S_1$, $r = 0$. TD target: $0 + 0.9V(S_1)$. Nếu $V(S_1) = 0.5$, cập nhật $V(S_0)$ ngay dựa trên 0.45 (không đợi episode kết thúc).

Ví dụ 3 — Cliff Walking với SARSA vs Q-Learning:

Layout: agent đi từ start (bottom-left) đến goal (bottom-right). Giữa có cliff (vách đá): step vào $\rightarrow r = -100$, reset về start.

SARSA với ϵ -greedy ($\epsilon = 0.1$):

- Khi học, SARSA biết nó sẽ thỉnh thoảng explore (0.1 xác suất random) $\rightarrow$ có thể step vào cliff nếu đi sát.
- Vì update dựa trên a_{t+1} thực tế (có khả năng explore), SARSA học policy **an toàn**: đi xa cliff (đường dài hơn nhưng ít risk).
- $Q^{\text{SARSA}}(s_{\text{near cliff}}, \text{down}) < Q(s, \text{up})$ (penalty từ exploration mistakes).

Q-Learning với ϵ -greedy:

- Update dựa trên $\max_a Q(s', a)$ (giả định greedy).
- Q-Learning học policy **tối ưu theo greedy**: đi sát cliff (đường ngắn nhất).
- Nhưng trong execution với ϵ -greedy, vẫn có 0.1 xác suất explore $\rightarrow$ đôi khi rơi xuống cliff.
- $Q^*(s_{\text{near cliff}}, \text{down})$ cao (vì assume greedy không explore).

Return: Q-Learning học policy tốt hơn (theo lý thuyết), nhưng SARSA an toàn hơn trong quá trình training với exploration noise.

Tài liệu tham khảo:

- *Sutton & Barto (2nd ed.)*, Chapter 5 (Monte Carlo), Chapter 6 (TD Learning, SARSA, Q-Learning) — giải thích chi tiết và so sánh.
- *David Silver Lecture 4 & 5* — Model-Free Prediction và Control.
- *Denny Britz RL GitHub repo* (<https://github.com/dennybritz/reinforcement-learning>) — code Python minh họa MC, TD, SARSA, Q-Learning.

4.7 Python, NumPy và môi trường RL — cần cho thực hành

Bạn nên có khả năng:

- Viết vòng lặp `for`, `while`.
- Dùng hàm và class cơ bản.
- Làm việc với NumPy array.
- Lập chỉ mục ma trận.

- Sampling ngẫu nhiên.
- Vẽ reward theo episode.
- Đọc API của môi trường.

Công cụ thực hành phù hợp:

- Python.
- NumPy.
- Matplotlib.
- Gymnasium.
- PyTorch khi chuyển sang Deep RL.

Không nên chỉ học Q-Learning bằng Matlab nếu mục tiêu dài hạn là nghiên cứu hoặc làm dự án AI hiện đại.

Lý thuyết cơ bản:

Để implement RL algorithms, cần:

- **NumPy**: lưu Q-table, V-table dưới dạng array, thao tác ma trận.
- **Gymnasium**: thư viện môi trường RL chuẩn (kế thừa OpenAI Gym).
- **Matplotlib**: vẽ learning curve (reward theo episode).

Gymnasium API:

```
env = gym.make("FrozenLake-v1")
observation, info = env.reset()
action = env.action_space.sample() # hoặc chọn theo policy
observation, reward, terminated, truncated, info = env.step(action)
done = terminated or truncated
```

Q-table: ma trận 2D $Q[s, a]$ với kích thước $|\mathcal{S}| \times |\mathcal{A}|$. Khởi tạo:

```
Q = np.zeros([n_states, n_actions])
# hoặc optimistic initialization:
Q = np.ones([n_states, n_actions]) * 5.0
```

Ví dụ 1 — Code Q-Learning với Gymnasium:

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt
```

```

env = gym.make("FrozenLake-v1", is_slippery=False)
Q = np.zeros([env.observation_space.n, env.action_space.n])
alpha = 0.1    # learning rate
gamma = 0.99   # discount factor
epsilon = 1.0
epsilon_decay = 0.995
epsilon_min = 0.01
n_episodes = 10000

rewards_per_episode = []

for episode in range(n_episodes):
    s, _ = env.reset()
    done = False
    total_reward = 0

    while not done:
        # Epsilon-greedy action selection
        if np.random.rand() < epsilon:
            a = env.action_space.sample() # explore
        else:
            a = np.argmax(Q[s]) # exploit

        s_next, r, terminated, truncated, _ = env.step(a)
        done = terminated or truncated

        # Q-Learning update
        Q[s, a] += alpha * (r + gamma * np.max(Q[s_next]) - Q[s, a])

        s = s_next
        total_reward += r

    # Decay epsilon
    epsilon = max(epsilon_min, epsilon * epsilon_decay)
    rewards_per_episode.append(total_reward)

# Plot learning curve
plt.plot(rewards_per_episode)
plt.xlabel('Episode')
plt.ylabel('Total Reward')
plt.show()

# Optimal policy
policy = np.argmax(Q, axis=1)
print("Optimal policy:", policy)

```

Ví dụ 2 — SARSA implementation:

Khác biệt chính: SARSA cần chọn a' trước khi update, Q-Learning dùng $\max_a Q(s', a)$.

```
# Inside episode loop, after reset:
a = epsilon_greedy_action(Q, s, epsilon) # chọn a đầu tiên

while not done:
    s_next, r, terminated, truncated, _ = env.step(a)
    done = terminated or truncated

    a_next = epsilon_greedy_action(Q, s_next, epsilon) # chọn a'

    # SARSA update (dùng a' thực tế, không phải max)
    Q[s, a] += alpha * (r + gamma * Q[s_next, a_next] - Q[s, a])

    s, a = s_next, a_next
```

Ví dụ 3 — Môi trường phổ biến cho RL cơ bản:

- **FrozenLake-v1**: 4×4 grid, tránh holes, đến goal. Discrete state/action. Tốt cho tabular Q-Learning.
- **Taxi-v3**: taxi pick up và drop off passenger. 500 states, 6 actions.
- **CliffWalking-v0**: cliff walking example, so sánh SARSA vs Q-Learning.
- **CartPole-v1**: cân bằng cột trên xe. Continuous state (4D), discrete action (2). Cần function approximation hoặc discretization.

Tài liệu tham khảo:

- *Gymnasium Documentation* (<https://gymnasium.farama.org/>) — thư viện môi trường RL chuẩn (kế thừa OpenAI Gym).
- *NumPy Quickstart Tutorial* (<https://numpy.org/doc/stable/user/quickstart.html>) — nền tảng NumPy cho RL.
- *Python Data Science Handbook* (Jake VanderPlas) — Python, NumPy, Matplotlib.
- *Denny Britz RL repo* — code mẫu Q-Learning với Gymnasium/Gym.

5 Điều kiện để chuyển từ RL cơ bản sang Deep RL

Sau khi làm được Q-Learning bằng Q-table, bạn mới nên chuyển sang DQN.

Trước DQN cần hiểu:

- Function approximation.
- Regression với neural network.
- Mean squared error.

- Gradient descent.
- Input state và output Q-values.
- Replay buffer.
- Mini-batch sampling.
- Target network.
- Temporal-difference target.
- Vì sao huấn luyện neural network trực tiếp trên dữ liệu RL dễ mất ổn định.

Bạn đã có phần lớn nền tảng neural network cho đoạn này. Phần còn thiếu chủ yếu là **kiến thức RL**, không phải kiến thức CNN.

Sau DQN mới nên học:

- Policy Gradient.
- REINFORCE.
- Actor–Critic.
- Advantage Actor–Critic.
- PPO.

6 Lộ trình phù hợp nhất với tình trạng hiện tại

6.1 Giai đoạn 1 — Bổ sung nền tảng tối thiểu

Học:

1. Supervised, unsupervised và reinforcement learning khác nhau thế nào.
2. Linear regression, loss MSE và function approximation.
3. Xác suất, kỳ vọng và sampling.
4. Khái niệm Markov property.

Không cần chờ hoàn thành KNN, K-Means và SVM.

6.2 Giai đoạn 2 — RL dạng bảng

Học theo thứ tự:

1. Agent–environment interaction.
2. State, action, reward, policy.
3. Return và discount factor.
4. MDP.
5. State-value và action-value.
6. Bellman equation.
7. Policy iteration và value iteration.
8. Monte Carlo.
9. TD learning.
10. SARSA.
11. Q-Learning.

Thực hành:

- Multi-armed bandit.
- GridWorld.
- FrozenLake.
- Taxi.

6.3 Giai đoạn 3 — Deep RL

Sau khi tự cài đặt được Q-Learning:

1. Function approximation.
2. DQN.
3. Experience replay.
4. Target network.
5. Double DQN.
6. Policy Gradient.
7. Actor–Critic.
8. PPO.

Thực hành ban đầu với CartPole và LunarLander; chưa cần dùng ảnh nên chưa cần CNN.

7 Checklist sẵn sàng

Bạn có thể bắt đầu Q-Learning khi trả lời được các câu sau:

- State, action và reward khác nhau thế nào?
- Policy là gì?
- Episode kết thúc khi nào?
- Return khác reward tức thời thế nào?
- Discount factor γ có tác dụng gì?
- Kỳ vọng toán học là gì?
- $V(s)$ khác $Q(s, a)$ thế nào?
- Bellman equation thể hiện ý tưởng gì?
- Exploration khác exploitation thế nào?
- Learning rate α điều khiển điều gì?

Đánh giá hiện tại: bạn đã sẵn sàng bắt đầu phần nhập môn RL, nhưng nên dành một chặng ngắn bổ sung **xác suất + MDP + Bellman equation** trước hoặc học song song với nội dung Q-Learning. CNN và AlexNet hiện tại đã nhiều hơn mức cần thiết cho RL cơ bản.